

Volume 00

WarrantyGraph

Master Index

Reading map for the multi-volume technical library

Full project library

This multi-volume set covers the entire repository — not a single session. Algorithms, theory, annotated code, RDF/OWL, pipelines, and deploy are all included.

1. Volume map

Vol	Title	Read when you need...
00	Master Index	Orientation
01	Architecture & System Map	Layers, packages, APIs, network
02	Algorithms & Theory	FMEA, Bayes, TF-IDF, GraphRAG math
03	Code Deep-Dive	Real code + what it does + why
04	Ontology RDF OWL Turtle	Formal schema & serializations
05	Pipelines Deploy Tests Data	ETL, k8s, evals, fixtures

2. Recommended reading orders

- **New engineer:** 01 → 03 (services+graph_rag) → 05 runbook → 02 algorithms
- **Data scientist / academic:** 02 first → 04 ontology → 03 reliability.py
- **Architect:** 01 → 05 deploy → 02 ranking design → 04 schema contract
- **Interview prep:** 02 theory whiteboard → 03 diagnose path → docs/interview PDF

3. Core idea in one paragraph

WarrantyGraph turns appliance failure knowledge into a **Neo4j property graph**. At runtime it matches free-text symptoms, ranks failure modes with **FMEA + naive Bayes**, walks diagnostic trees and BOM edges for parts, and returns explainable confidence with provenance. Knowledge is ETL'd from enterprise-like sources; LLM is optional and off by default.

4. Ports & entrypoints

Service	Port / command
Next.js UI	:3000 — frontend/ npm run dev
FastAPI	:8080 — uvicorn api.main:app
Neo4j	:7687 Bolt — populate_graph.py
Mock enterprise	:8090 — simulation.mock_enterprise_apps
Redis optional	:6379 — REDIS_URL
ETL spine	python -m graph.enterprise_pipeline.orchestrator

5. How every topic is documented (WWWH)

Across volumes we use a fixed lens: **What** (plain English) · **Where** (file/package) · **When** (batch vs request) · **How** (mechanism) · **Why** (design reason). Indexes: see Vol 05 §Indexes and docs/19-Indexes-Constraints-and-Lookup-Performance.md.

WHAT	Uniqueness constraints + SQLite PKs/status indexes
WHERE	populate_graph.create_constraints · utils/persistence.py
WHEN	Constraints on every graph load; SQLite on first ops DB open
HOW	Neo4j UNIQUE on *_id; SQLite PRIMARY KEY + idx_*_status
WHY	Fast id seeks, no duplicate entities, agent list filters